

Data Analysis and Visualization

Implement Data Loading, Storage and File Formats. Read data and store them in text format.

We have used **inbuilt python dictionary** to create a dataset, and then with the help of **pandas** library we have save the file in various format.

Decision (Store not Store)

To which to store and when not to use none test

1. assign the change to the variable
2. print it

if printed then store if none then no need .

Task 1 "Workflow: Creating, Saving, and Verifying Tab-Separated Text Files"

Pandas - From Dictionary to Text File.

1. Create a Sample Data
2. Create a DataFrame
3. Define the Destination Address
4. Send df to csv text file
5. Read Data Back from the Text File (Verification Step)

In a real data science project, you usually have two different scripts: Script A: Collects data and saves it to a file. Script B: Opens that file weeks later to analyze it.

```
In [18]: # Task 1 Full Reference
import pandas as pd

# =====
# PHASE 1: DATA CREATION (The "Box")
# =====
# We define the raw data dictionary in memory.
data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [25, 30, 35],
    "City": ["New York", "Los Angeles", "Chicago"]
}

# We load that data into a DataFrame object.
```

```

# 'df' is now a container holding our data in RAM.
df = pd.DataFrame(data)

# =====
# PHASE 2: CONFIGURATION (The "Address")
# =====
# CRITICAL CORRECTION: This step does NOT save the file.
# We are simply creating a variable to store the name/location
# of the destination file. It is just a string of text right now.
text_file_path = "data.txt"

# =====
# PHASE 3: EXECUTION (The "Shipping")
# =====
# This is the "Action" step.
# We connect the Data (df) to the Destination (text_file_path).
# - sep='\t' : Use tabs to separate columns (instead of commas).
# - index=False: Do not write row numbers (0, 1, 2) to the file.
df.to_csv(text_file_path, index=False, sep='\t')

print(f"Success: File has been physically created at '{text_file_path}'")

# =====
# PHASE 4: VERIFICATION (The "Check")
# =====
# We read the file back from the hard drive to prove it worked.
# Note: We MUST tell pandas to expect tabs (sep='\t') again,
# otherwise it will try to read commas and fail.
df_check = pd.read_csv(text_file_path, sep='\t')

print("\n--- Verification: Data read back from file ---")
print(df_check)

```

Success: File has been physically created at 'data.txt'

--- Verification: Data read back from file ---

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

```

In [16]: # Create a Sample Data
data = {"Name": ["Alice", "Bob", "Charlie"], "Age": [25, 30, 35], "City": ["New York"]
# Create the dataframe
df = pd.DataFrame(data)
# Create text file nametag
text_file_path = "data.txt"
# transfer the data to tab seperated values file format
df.to_csv(text_file_path, index=False, sep="\t")
# print the success
print(f"Name of the file where Data saved in TSV file: {text_file_path}")
# Verification step
read_tsv_df = pd.read_csv(text_file_path, sep="\t")

```

```
print("Read the TSV file ")
print(read_tsv_df)
```

Name of the file where Data saved in TSV file: data.txt

Read the TSV file

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

Note

1. The Method is Flexible to_csv is misleading: It is not just for Comma Separated Values. It is a general engine for "Writing Flat Text Files." File Extension: You can name the file .txt, .csv, or anything else; Pandas doesn't care.
2. The Parameters Control the Format sep='\t': This overrides the default comma. It inserts an invisible Tab character between data points.index=False: This removes the row numbers (0, 1, 2...) from the far left, keeping the file clean.
3. Alignment is Automatic No Math Required: You do not need to calculate spaces or word lengths.The "Tab" Job: The \t character tells your text editor (Notepad, Excel, etc.) to jump to the next alignment point automatically.

Task 2 Save and Read in Other format # from step 4 in practical 1 DV journal

TO JSON - Javascript Object Notation

In [20]: data

```
Out[20]: {'Name': ['Alice', 'Bob', 'Charlie'],
          'Age': [25, 30, 35],
          'City': ['New York', 'Los Angeles', 'Chicago']}
```

```
In [19]: # Create the json file nametag
json_file_path = "data.json"
# tranfers the data to created json file format
df.to_json(json_file_path,orient="records",lines=True)
# print the success
print(f"Data saved in json file: {json_file_path}")
# read stored file
read_json_df = pd.read_json(json_file_path,orient="records",lines=True)
print(read_json_df)
```

Data saved in json file: data.json

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

Task 3 Save and Read in Excel format

TO Excel - Exsl

```
In [24]: # Correct file extension: .xlsx
excel_file_path = "data.xlsx"

# Transfer the data to the created Excel file
df.to_excel(excel_file_path, index=False)

print(f>Data saved in Excel file: {excel_file_path}")

# Read stored file
read_excel_df = pd.read_excel(excel_file_path)
print(read_excel_df)
```

Data saved in Excel file: data.xlsx

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

Data Wrangling

```
In [25]: import pandas as pd

# --- Step 1: Create a Complex Dataset ---
data = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Eva"],
    "Age": [25, 30, 35, 40, None],
    "City": ["New York", "Los Angeles", "Chicago", "Houston", "Phoenix"],
    "Salary": [70000, 80000, 120000, 110000, None],
    "Department": ["HR", "Finance", "Engineering", "Management", "HR"]
}

# Create a DataFrame
df = pd.DataFrame(data)

# --- Step 2: Data Cleaning ---
# Handle missing values (Using assignment instead of inplace=True for future compat
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Salary"] = df["Salary"].fillna(df["Salary"].median())

# Add a new calculated column
df["Tax"] = df["Salary"] * 0.2
```

```

# --- Step 3: Save Data in Multiple Formats ---

# Save as CSV
csv_file_path = "complex_data.csv"
df.to_csv(csv_file_path, index=False)
print(f>Data saved as CSV at: {csv_file_path}")

# Save as JSON (using the format we discussed previously)
json_file_path = "complex_data.json"
df.to_json(json_file_path, orient='records', lines=True)
print(f>Data saved as JSON at: {json_file_path}")

# --- Step 4: Read Data Back and Perform Analysis ---
# Read from CSV
read_csv_df = pd.read_csv(csv_file_path)
print("\n--- Data read from CSV ---")
print(read_csv_df)

# Read from JSON
read_json_df = pd.read_json(json_file_path, orient='records', lines=True)
print("\n--- Data read from JSON ---")
print(read_json_df)

# --- Step 5: Advanced Analysis ---

# Group by Department and Calculate Mean Salary
grouped_df = df.groupby("Department")["Salary"].mean().reset_index()
print("\n--- Average Salary by Department ---")
print(grouped_df)

# Save Grouped Data as Excel
excel_file_path = "grouped_data.xlsx"
grouped_df.to_excel(excel_file_path, index=False)
print(f\nGrouped data saved as Excel at: {excel_file_path}")

```

Data saved as CSV at: complex_data.csv
Data saved as JSON at: complex_data.json

--- Data read from CSV ---

	Name	Age	City	Salary	Department	Tax
0	Alice	25.0	New York	70000.0	HR	14000.0
1	Bob	30.0	Los Angeles	80000.0	Finance	16000.0
2	Charlie	35.0	Chicago	120000.0	Engineering	24000.0
3	David	40.0	Houston	110000.0	Management	22000.0
4	Eva	32.5	Phoenix	95000.0	HR	19000.0

--- Data read from JSON ---

	Name	Age	City	Salary	Department	Tax
0	Alice	25.0	New York	70000	HR	14000
1	Bob	30.0	Los Angeles	80000	Finance	16000
2	Charlie	35.0	Chicago	120000	Engineering	24000
3	David	40.0	Houston	110000	Management	22000
4	Eva	32.5	Phoenix	95000	HR	19000

--- Average Salary by Department ---

	Department	Salary
0	Engineering	120000.0
1	Finance	80000.0
2	HR	82500.0
3	Management	110000.0

Grouped data saved as Excel at: grouped_data.xlsx

In []: *# Practical 1 Steps (11)*

1. **Library Import**
2. **Dataset Definition**
3. **DataFrame Creation**
4. **Data Cleaning (Imputation)**
5. **Feature Engineering (Calculation)**
6. **CSV Export (Save)**
7. **JSON Export (Save)**
8. **CSV Import (Read)**
9. **JSON Import (Read)**
10. **Grouped Aggregation (Analysis)**
11. **Excel Export**

In [46]: `import pandas as pd`

```
c_data = {"Name":["Mave","Ryan","James","Hellen","Clark"],"Age":[27,28,29,26,None],
```

In [47]: `c_data`

```
Out[47]: {'Name': ['Mave', 'Ryan', 'James', 'Hellen', 'Clark'],
          'Age': [27, 28, 29, 26, None],
          'City': ['Tokyo', 'Berlin', 'Milan', 'Prague', 'Madrid'],
          'Salary': [55000, 77000, 22000, 11000, None],
          'Department': ['R&D', 'IT', 'HR', 'CSR', 'IT']}
```

```
In [48]: df1 = pd.DataFrame(c_data)
df1
```

```
Out[48]:
```

	Name	Age	City	Salary	Department
0	Mave	27.0	Tokyo	55000.0	R&D
1	Ryan	28.0	Berlin	77000.0	IT
2	James	29.0	Milan	22000.0	HR
3	Hellen	26.0	Prague	11000.0	CSR
4	Clark	NaN	Madrid	NaN	IT

```
In [58]: # data cleaning
df1["Age"] = df1["Age"].fillna(df1["Age"].mean())
df1["Salary"] = df1["Salary"].fillna(df1["Salary"].median())

# Feature engineering
df1["Tax"] = df1["Salary"] * 0.2

# Create the csv file address
csv_file_p = "c_data.csv"
# transfer data to csv file (Save data)
df1.to_csv(csv_file_p, index=False)
print(f"Data saved in Csv file: {csv_file_p}")
print("-----")

# Create json file address
json_file_p = "j_data.json"
# transfer the data to json file (Save data)
df1.to_json(json_file_p, orient="records", lines=True)
print(f"Data saved in json file: {json_file_p}")
print("-----")

# Read data from CSV file
read_csv = pd.read_csv(csv_file_p)
print("-----CSV File -----")
print(read_csv)
print("-----")

# Read data from Json file
read_json = pd.read_json(json_file_p, orient="records", lines=True)
print("----- Json File -----")
print(read_json)

# Advance Analytics - (Groupby)
grouped_data = df1.groupby("Department")["Salary"].mean().reset_index()
```

```

print("-----Average Salary by Department-----")

# Transfer grouped data to the excel file
excel_grouped_data = "g_data.xlsx"
grouped_data.to_excel(excel_grouped_data,index=False)
print("-----")
print(f" Grouped Data saved in Excel file:{excel_grouped_data}")

# Read the grouped data
read_grouped_data = pd.read_excel(excel_grouped_data)
print(read_grouped_data)

```

Data saved in Csv file: c_data.csv

Data saved in json file: j_data.json

-----CSV File -----

	Name	Age	City	Salary	Department	Tax
0	Mave	27.0	Tokyo	55000.0	R&D	11000.0
1	Ryan	28.0	Berlin	77000.0	IT	15400.0
2	James	29.0	Milan	22000.0	HR	4400.0
3	Hellen	26.0	Prague	11000.0	CSR	2200.0
4	Clark	27.5	Madrid	38500.0	IT	7700.0

----- Json File -----

	Name	Age	City	Salary	Department	Tax
0	Mave	27.0	Tokyo	55000	R&D	11000
1	Ryan	28.0	Berlin	77000	IT	15400
2	James	29.0	Milan	22000	HR	4400
3	Hellen	26.0	Prague	11000	CSR	2200
4	Clark	27.5	Madrid	38500	IT	7700

-----Average Salary by Department-----

Grouped Data saved in Excel file:g_data.xlsx

	Department	Salary
0	CSR	11000
1	HR	22000
2	IT	57750
3	R&D	55000



dot pd for read_csv/json/excel

In []: